

CSCI 232

232 Algorithms & Data Structures

LECTURE: 10

J. L. Pach

TODAY'S AGENDA

- An algorithm
- NP vs P
- Bubble sort

ALGORITHM

ALGORITHM

- Informally, an **algorithm** is any well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**. An algorithm is thus a sequence of computational steps that transform the input into the output.
- We can also view an algorithm as a tool for solving a well-specified **computational problem**. The statement of the problem specifies in general terms the desired input/output relationship. The algorithm describes a specific computational procedure for achieving that input/output relationship.

ALGORITHM

For example, we might need to sort a sequence of numbers into nondecreasing order. This problem arises frequently in practice and provides fertile ground for introducing many standard design techniques and analysis tools. Here is how we formally define the sorting problem:

Input: A sequence of n numbers $< a_1, a_2, \ldots, a_n >$.
Output: A permutation (reordering) $< a'_1, a'_2, \ldots, a'_n >$. of the input sequence such that $a'_1 < a'_2 < \ldots < a'_n$.

For example, given the input sequence $< 31; 41; 59; 26; 41; 58 >$, a sorting algorithm returns as output the sequence $< 26; 31; 41; 41; 58; 59 >$.

ALGORITHM

Such an input sequence is called an **instance** of the sorting problem. In general, an **instance of a problem** consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.

An algorithm is said to be **correct** if, for every input instance, it halts with the correct output. We say that a correct algorithm solves the given computational problem.

An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer.

PRACTICAL APPLICATIONS OF ALGORITHMS:

- The Human Genome Project has made great progress toward the goals of identifying all the 100,000 genes in human DNA, determining the sequences of the 3 billion chemical base pairs that make up human DNA.
- The Internet enables people all around the world to quickly access and retrieve large amounts of information. With the aid of clever algorithms, sites on the Internet are able to manage and manipulate this large volume of data.
- Electronic commerce enables goods and services to be negotiated and exchanged electronically, and it depends on the privacy of personal information such as credit card numbers, passwords, and bank statements.

PRACTICAL APPLICATIONS OF ALGORITHMS:

- Manufacturing and other commercial enterprises often need to allocate scarce resources in the most beneficial way.
- We are given a road map on which the distance between each pair of adjacent intersections is marked, and we wish to determine the shortest route from one intersection to another.

TWO CHARACTERISTICS THAT ARE COMMON TO MANY INTERESTING ALGORITHMIC PROBLEMS:

1. They have many candidate solutions, the overwhelming majority of which do not solve the problem at hand. Finding one that does, or one that is “best,” can present quite a challenge.
2. They have practical applications. Of the problems in the above list, finding the shortest path provides the easiest examples. A transportation firm, such as a trucking or railroad company, has a financial interest in finding shortest paths through a road or rail network because taking shorter paths results in lower labor and fuel costs.

DATA STRUCTURES

We have got several data structures. **A data structure** is a way to store and organize data in order to facilitate access and modifications. No single data structure works well for all purposes, and so it is important to know the strengths and limitations of several of them.

HARD PROBLEMS

- Most of this course is about efficient algorithms. Our usual measure of efficiency is speed, i.e., how long an algorithm takes to produce its result. There are some problems, however, for which no efficient solution is known.
- An interesting subset of these problems, which are known as NP-complete. (nondeterministic polynomial-time complete).
- Why are NP-complete problems interesting? First, although no efficient algorithm for an NP-complete problem has ever been found, nobody has ever proven that an efficient algorithm for one cannot exist.

NP VS P

NP VS P

- NP problems are those for which the correctness of a given **solution can be verified in polynomial time**.
- P problems, on the other hand, **can be solved in polynomial time**.
- Imagine having a riddle. Finding the solution to this riddle might be very difficult and time-consuming. However, if someone gives you a solution, you can quickly verify whether it is correct. This is somewhat analogous to NP problems: finding a solution can be hard, but verifying a given solution is easy.

NP-COMPLETE EXAMPLE

As a concrete example, consider a delivery company with a central depot. Each day, it loads up each delivery truck at the depot and sends it around to deliver goods to several addresses. At the end of the day, each truck must end up back at the depot so that it is ready to be loaded for the next day. To reduce costs, the company wants to select an order of delivery stops that yields the lowest overall distance traveled by each truck. This problem is the well-known "**traveling-salesman problem**," and it is **NP-complete**. It has no known efficient algorithm. Under certain assumptions, however, we know of efficient algorithms that give an overall distance which is not too far above the smallest possible.

ALGORITHM

notation systems

AN ALGORITHM CAN BE REPRESENTED IN AT LEAST THREE WAYS:

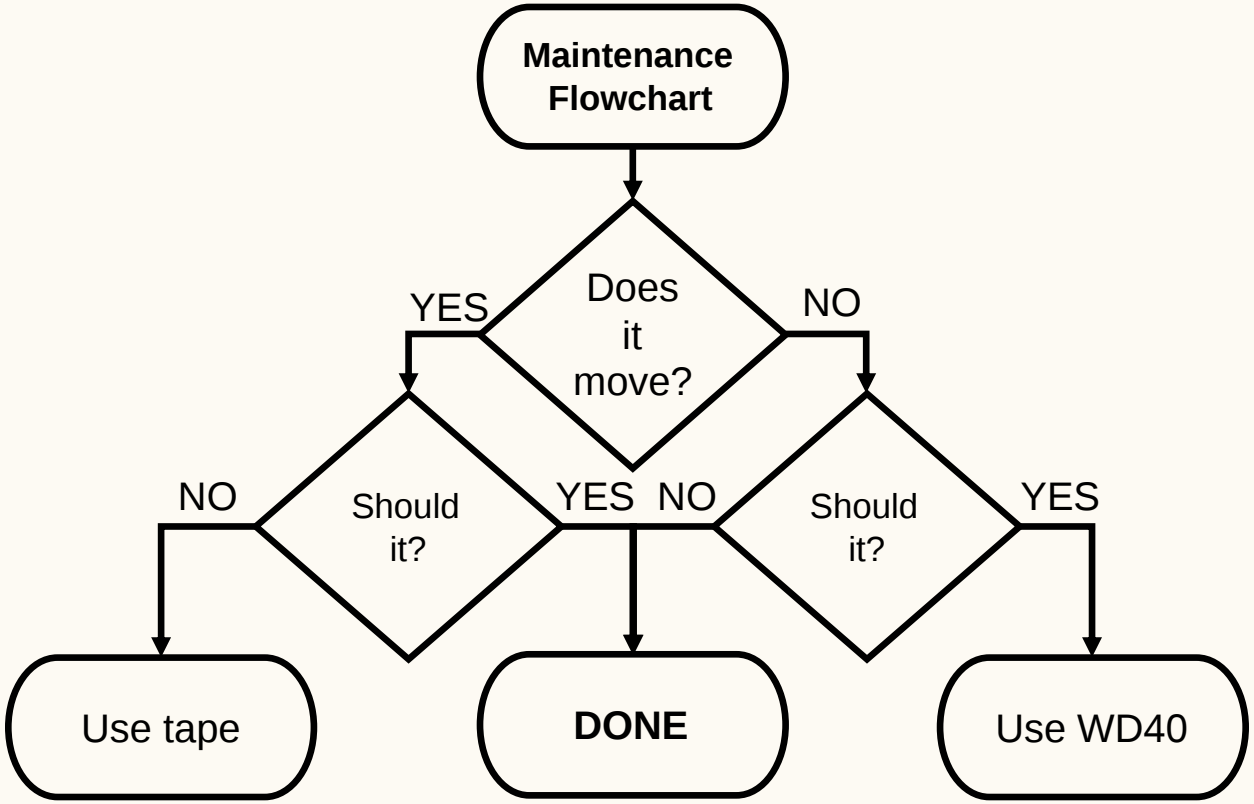
- **Flowchart:**
  - This involves breaking down the algorithm into elementary control blocks such as start, stop, data input, output, conditional statements, etc.
- **Bullet point description:**
  - The individual steps are described verbally using bullet points. The points are numbered to allow for a clear description of the method's flow.
- **Pseudocode:**
  - Pseudocode resembles a programming language like Pascal*, but only uses keywords such as var, begin, end, for, etc."

PSEUDOCODE RESEMBLES A PROGRAMMING LANGUAGE LIKE PASCAL*

When the C language was being born, Pascal was already a popular language. It contributed significantly—or even profoundly—to the world of programming, but it did not stand the test of time. Languages derived from C eventually replaced it entirely. Today, Python is the most popular language for describing programming concepts due to its simplicity, so pseudocode will be a mixture of natural language and a specific programming language.

FLOWCHART

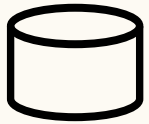
A flowchart is a graphical representation of an algorithm. It uses shapes to represent different types of operations and arrows to indicate the flow of control between them.



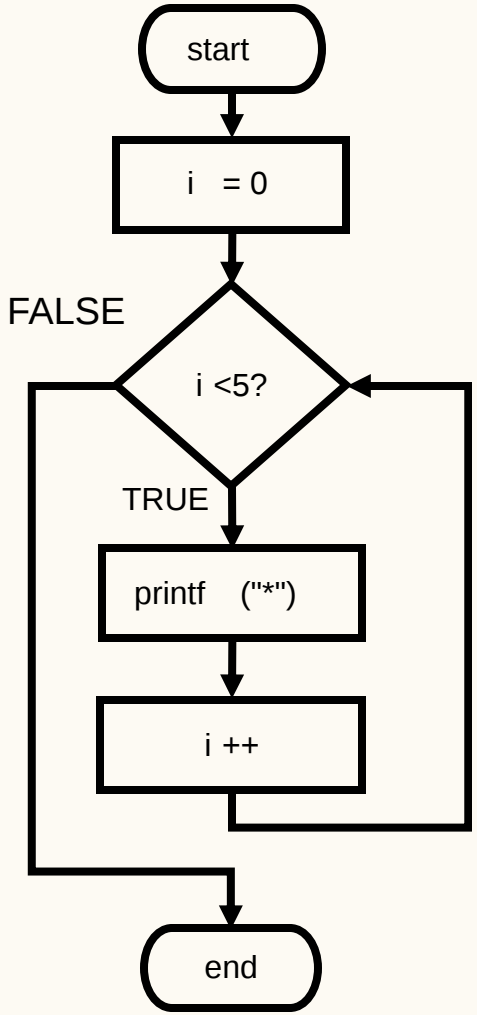
FLOWCHART SYMBOLS 1/2

Shape	Name	Description
	Flowline/Arrowhead	Shows the process's order of operation. A line coming from on symbol and pointing at another.
	Terminal	Indicates the beginning and ending of a program or sub process.
	Process	Represents a set of operations that changes value, form, or location of data.
	Decision	Shows a conditional operation that determines which one of the two paths the program will take.
	Input/Output	Indicates the process of inputting and outputting data.

FLOWCHART SYMBOLS 2/2

Shape	Name	Description
	Predefined Process/ A Black Box	Shows named process which is defined elsewhere.
	(On-page) Connector	Pairs of labeled connectors replace long or confusing lines on a flowchart page.
	(Off-page) Connector	A labeled connector for use when the target is on another page.
	Data File or Database	Data represented by a cylinder symbolizing ~~a disk drive~~ any kind...

FLOW DIAGRAM A C-STYLE FOR LOOP, REPRESENTING THE FOLLOWING CODE:




```c
for ( i = 0; i < n; i++)
{
        printf("*");
}
```

BULLET POINT DESCRIPTION

Input:

* N: Number of elements in the array, integer type
* Arr: Array of double type

Output:

* Maximum: Variable of double type

Method:

1. Initialize Maximum to the first element of the array (Arr[0]) and set Index to 0.
2. Increment Index by 1.
3. If Index is less than N, go to step 4. Otherwise, proceed to step 6.
4. If Maximum is less than the element at the current index (Arr[Index]), update Maximum to the value at Arr5[Index].
5. Go back to step 2.
6. Print the Maximum value and end the program.

PSEUDOCODE (PASCAL*)

```
function factorial(n: integer): integer;
begin
 if n = 0 then
    factorial := 1
else
    factorial := n * factorial(n - 1);
end;
```

Advantages and Disadvantages of Flowchart Algorithm Representation

- Advantages:
  - **Visualization**: Presents the algorithm in a graphical manner, making it easier to understand, especially for visual learners.
  - **Intuitiveness**: Arrows and shapes clearly show the sequence of actions.
  - **Universality**: Can be used to represent various types of algorithms, regardless of the programming language.
- Disadvantages:
  - **Complexity**: For large and complex algorithms, the diagram can become unreadable.
  - **Time-consuming**: Creating a detailed flowchart can be time-consuming.
  - **Limitations in expressing certain concepts**: It's not always easy to represent more abstract concepts in a flowchart.

Advantages and Disadvantages of Bullet point description of Algorithm Representation

- Advantages:
  - **Simplicity**: Easy to understand, even for beginners.
  - **Flexibility**: Natural language can be used, allowing for more detailed explanations.
  - **Focus on the idea**: Allows you to focus on the main idea of the algorithm, rather than implementation details.
- Disadvantages:
  - **Lack of formality**: Not as precise as pseudocode or a flowchart.
  - **Potential ambiguity**: Interpretation of individual points may vary depending on the reader.
  - **Limitations for complex algorithms**: For more complex algorithms, a bullet point list may be insufficient.


Picture 4

Advantages and Disadvantages of Pseudocode of Algorithm Representation

- Advantages:
  - **Intermediate between natural language and programming language**: Easier to understand than pure code, but more formal than a bullet point description.
  - **Precision**: Allows for a precise description of the algorithm's steps.
  - **Programming language independence**: Can be easily* translated into any programming language.
- Disadvantages:
  - **Less intuitive than a flowchart**: Requires more effort to understand the algorithm's logic.
  - **Possibility of errors**: If the pseudocode is not written carefully, it may contain inaccuracies.

THANK

You

OUTLINE

- Primitive operations — the unit of measurement
- Counting primitive operations in code
- From a count to T(n)
- From T(n) to asymptotic notation

PRIMITIVE

Operations

WHAT IS A PRIMITIVE OPERATION?

To measure the running time of an algorithm, we need a **unit of measurement** that is independent of the hardware, the compiler, and the programming language.

We use **primitive operations** — basic computational steps that take a constant amount of time, regardless of the input size.

> A primitive operation always costs the same — O(1).
> We count how many of them an algorithm performs.

THE SEVEN PRIMITIVE OPERATIONS

Symbol	Operation	Example
c1	Assigning a value to a variable	`x = 5`
c2	Calling a method / function	`sort(arr)`
c3	Performing an arithmetic operation	`a + b`, `a * b`
c4	Comparing two numbers	`a > b`, `a == b`
c5	Indexing into an array	`arr[i]`
c6	Following an object reference	`node->next`
c7	Returning from a method	`return x`

WHY THESE SEVEN?

Each of these operations takes a **fixed, constant amount of time** on any real computer.

It does not matter whether:

- the variable holds the value 1 or 1,000,000
- the array has 10 elements or 10 million elements
- the comparison is true or false

The time to execute one primitive operation is always bounded by a constant.

> This is why we can count them and use the count as a proxy for running time.

EACH OPERATION — ONE STEP

```c
int x = 5;
```



Step	Operation	Cost
assign 5 to x	**c1** — assigning a value	1

```c
int result = a + b;
```



Step	Operation	Cost
perform a + b	**c3** — arithmetic operation	1
assign result to variable	**c1** — assigning a value	1
Total		**2**

```
if (a > b)
```



Step	Operation	Cost
compare a and b	**c4** — comparing two numbers	1
Total		**1**

ARRAY ACCESS 1/2

```c
int x = arr[i];
```



Step	Operation	Cost
index into arr at position i	**c5** — indexing into array	1
assign value to x	**c1** — assigning a value	1
Total		**2**

Array Access 2/2

```c
int x = arr[i] + arr[j];
```



Step	Operation	Cost
index arr[i]	**c5**	1
index arr[j]	**c5**	1
perform addition	**c3**	1
assign to x	**c1**	1
Total		**4**

FUNCTION CALL AND RETURN

```c
int add(int a, int b)
{
    return a + b;
}

int result = add(x, y);
```



Step	Operation	Cost
call add(x, y)	**c2** — calling a method	1
perform a + b	**c3** — arithmetic	1
return from add	**c7** — returning from method	1

COUNTING OPERATIONS

in a Loop

A SIMPLE LOOP

```
int sum = 0;                    // c1 — 1 time
for (int i = 0; i < n; i++)     // c1 + n*(c4 + c3 + c1)
{
    sum = sum + arr[i];         // n * (c5 + c3 + c1)
}
return sum;                     // c7 — 1 time
```



Let us count every step:

Line	Operation	Count
`int sum = 0`	c1	1
`int i = 0`	c1	1

A SIMPLE LOOP — TOTAL

$$T(n) = 1 + 1 + (n+1) + 2n + n + n + n + 1$$
$$T(n) = 4 + 6n$$

This is the **exact** count of primitive operations for this loop.

> For n = 10: T(10) = 64 operations
> For n = 100: T(100) = 604 operations
> For n = 1000: T(1000) = 6004 operations

Notice: as n grows, the constant 4 becomes irrelevant.

The term **6n dominates** — the function grows linearly.

A NESTED LOOP

```
for (int i = 0; i < n; i++)          // outer: runs n times
{
    for (int j = 0; j < n; j++)      // inner: runs n times for each i
    {
        sum = sum + arr[i] + arr[j];  // 2×c5 + 2×c3 + c1 = 5 ops
    }
}
```



Count the inner body first:

Operation	Per inner iteration	Total inner iterations
`arr[i]` — c5	1	n × n
`arr[j]` — c5	1	n × n

NESTED LOOP — FULL COUNT

```
int sum = 0;                    // c1          →  1
for (int i = 0; i < n; i++)       // init + check + increment
{
    for (int j = 0; j < n; j++)     // init + check + increment
    {
        sum = sum + arr[i] + arr[j]; // 5 ops per iteration
    }
}
return sum;                     // c7          →  1
```

$$T(n) = 1 + 1 + (n+1) + n \cdot [1 + (n+1) + n + 5n] + 1$$

$$T(n) = 1 + 1 + (n+1) + n \cdot (7n+2) + 1$$

$$T(n) = 7n^2 + 3n + 4$$

The **n² term dominates** for large n.

The terms 3n and 4 become negligible as n grows.

FROM COUNT

to T(n)

THE PATTERN

After counting primitive operations we always get a polynomial in n:

Algorithm	Exact T(n)	Dominant term
Single loop	6n + 4	**n**
Nested loop	$7n^2 + 3n + 4$	**n^2**
Triple nested loop	$an^3 + bn^2 + cn + d$	**n^3**
Binary search	a·log n + b	**log n**

The **dominant term** is the one that grows fastest as n increases.

WHY LOWER TERMS DISAPPEAR

$$T(n) = 7n^2 + 3n + 4$$

Let us see how each term contributes as n grows:

n	$7n^2$	$3n$	4	% of total that is NOT $7n^2$
10	700	30	4	4.6%
100	70,000	300	4	0.4%
1,000	7,000,000	3,000	4	0.04%
10,000	700,000,000	30,000	4	0.004%

CONSTANT COEFFICIENTS ALSO DISAPPEAR

Compare two algorithms:

$$T_A(n) = 7n^2 \qquad T_B(n) = 2n^2$$

Both grow as n^2. The constant factor (7 vs 2) depends on:

- how many primitive operations we counted per step
- the speed of the specific hardware
- the quality of the compiler

These are **not properties of the algorithm** — they are properties of the environment.

We care about the **shape** of growth, not the constant multiplier.

THE RULE — DROP EVERYTHING EXCEPT THE DOMINANT TERM

$$T(n) = 7n^2 + 3n + 4$$

Step 1: Keep only the dominant term: $7n^2$

Step 2: Drop the constant coefficient: n^2

Step 3: Express as asymptotic class: $T(n) = \Theta(n^2)$

> This works because for large n:
> $\frac{1}{2}n^2 \leq 7n^2 + 3n + 4 \leq 8n^2 \quad \text{for all } n \geq n_0$
> which is exactly the Θ definition with $c_1 = ½$, $c_2 = 8$.

FROM T(N)

to Θ, O, Ω

CONNECTING T(N) TO THE THREE NOTATIONS

Given $T(n) = 7n^2 + 3n + 4$, we can say:

Notation	Meaning	Statement
Θ(n²)	tight bound — grows exactly like n²	$T(n) = \Theta(n^2)$
O(n²)	upper bound — grows no faster than n²	$T(n) = O(n^2)$
Ω(n²)	lower bound — grows no slower than n²	$T(n) = \Omega(n^2)$

All three are true here. But:

- **Θ is the most informative** — it says both above and below

THE FULL WORKFLOW

1. Write the algorithm
   ↓
2. Count primitive operations (c1...c7)
   ↓
3. Express as T(n) — exact polynomial
   ↓
4. Keep only the dominant term
   ↓
5. Drop constant coefficients
   ↓
6. Express as Θ(f(n)), O(f(n)), or Ω(f(n))

WORKED EXAMPLE — END TO END

```c
int findMax(int arr[], int n)
{
    int max = arr[0];              // c5 + c1 = 2
    for (int i = 1; i < n; i++)   // c1 + n*(c4 + c3 + c1) = 1 + 3n
    {
        if (arr[i] > max)          // c5 + c4 = 2n
            max = arr[i];          // c5 + c1 = 2n  (worst case)
    }
    return max;                    // c7 = 1
}
```



$$T(n) = 2 + 1 + 3n + 2n + 2n + 1 = 7n + 4$$

Dominant term: $7n$

$$T(n) = \Theta(n)$$

Linear — one pass through the array. Makes sense.

Summary

Code structure	T(n) shape	Complexity class
Single statement	constant	$\Theta(1)$
Single loop	$an + b$	$\Theta(n)$
Nested loop (2×)	$an^2 + bn + c$	$\Theta(n^2)$
Nested loop (3×)	$an^3 + \ldots$	$\Theta(n^3)$
Halving loop (binary search)	$a \cdot logn + b$	$\Theta(logn)$
Loop + halving inside	$a \cdot nlogn + b$	$\Theta(nlogn)$

> **The shape of the loop structure determines the complexity class.**
>
> Constants and lower terms are always dropped.



ARRAY INDEXING — THE AGU RULE

Modern processors have a dedicated **Address Generation Unit (AGU)** — a separate hardware unit that computes memory addresses independently from the ALU. This means that a simple array offset does **not** consume an arithmetic operation.

ARRAY INDEXING — SIMPLE INDEX

When the index is a **simple variable or constant** — the AGU handles it in one step:

```
tab[i]   // c5 — AGU computes base + i*sizeof(type)
tab[0]   // c5 — constant offset, AGU handles it
*tab     // c6 — pointer dereference, same mechanism
```



> `tab[i]` and `*(tab+i)` compile to the **same machine instruction**.
> The offset is built into the instruction — no separate ALU operation.

ARRAY INDEXING — COMPOUND EXPRESSION

When the index contains **any compound expression**, the ALU must compute it **before** the AGU can perform the access:

```
tab[i+j]     // c3 + c5  →  2 ops  (ALU: i+j,  then AGU: tab[result])
tab[i-1]     // c3 + c5  →  2 ops  (ALU: i-1,  then AGU)
tab[i*2]     // c3 + c5  →  2 ops  (ALU: i*2,  then AGU)
tab[i+j+1]   // c3 + c3 + c5  →  3 ops  (ALU twice, then AGU)
```



> We count the inner expression first — then add c5 for the indexing.
> We do **not** assume compiler or CPU optimizations.

We apply the following **consistent, architecture-independent** rule:

Expression	Rule	Cost
`tab[i]`	simple variable — AGU only	**c5 = 1**
`tab[0]`	constant — AGU only	**c5 = 1**
`*tab`	pointer dereference — AGU	**c6 = 1**
`node->next`	object reference — AGU	**c6 = 1**
`tab[n-1]`	compound expression + index	**c3 + c5 = 2**
`tab[i+j]`	compound expression + index	**c3 + c5 = 2**
`tab[i*2]`	compound expression + index	**c3 + c5 = 2**
`tab[i+j+1]`	two expressions + index	**c3 + c3 + c5 = 3**

ARRAY INDEXING — WHY THIS RULE? 1/2

You might ask:

If the processor can sometimes handle `tab[i*2]` *in a single instruction*

(when the scale fits a power of 2), why do we count it as two operations?

- The answer is the **RAM model**.
- We analyze the algorithm from the source code, not from the generated assembly.

Array Indexing — Why This Rule? 2/2

We do not know:

- ⚒ what compiler is used.
- ⚒ what optimizations are applied,
- ⚒ what the target architecture is.

So we apply the **conservative rule**:

any expression inside [] that is not a simple variable or constant — count it separately as c3.

This gives us counts that are **consistent, predictable, and machine-independent** — which is exactly the purpose of the RAM model.

> **Rule:** Simple index → c5 = 1.
> Compound expression inside brackets → count the expression first, then add c5.

```
##########################################################################
```

TODAY'S AGENDA

- Algorithm Correctness
- The Random Access Machine (RAM) Model
- Primitive Operations — c_1 through c_7
- Counting Primitive Operations — arrayMax
- Best Case and Worst Case
- Average-case and Worst-case Analysis

ALGORITHM

Correctness

SEARCHING FOR THE MAXIMUM VALUE IN AN ARRAY

```c
int arrayMax(int A[], int n)
{
    int currentMax = A[0];
    for (int i = 1; i < n; i++)
    {
        if (currentMax < A[i])
            currentMax = A[i];
    }
    return currentMax;
}
```

```
Algorithm arrayMax(A, n):
  Input:  array A storing n ≥ 1 integers
  Output: the maximum element in A

  currentMax ← A[0]
  for i ← 1 to n - 1 do
      if currentMax < A[i] then
          currentMax ← A[i]
  return currentMax
```

Curio — Two Equivalent Implementations

```c
// Version 1 — explicit if statement
int arrayMax(int A[], int n)
{
    int currentMax = A[0];
    for (int i = 1; i < n; i++)
    {
        if (currentMax < A[i])
            currentMax = A[i];
    }
    return currentMax;
}
```

```c
// Version 2 — ternary operator
int arrayMax(int A[], int n)
{
    int currentMax = A[0];
    for (int i = 1; i < n; i++)
        currentMax = currentMax < A[i] ? A[i] : currentMax;
    return currentMax;
}
```

> Which is faster? We will answer this after counting primitive operations.

USING PSEUDOCODE TO PROVE ALGORITHM CORRECTNESS

By inspecting the pseudocode, we can argue about the correctness of `arrayMax` with a simple argument.

`currentMax` starts out equal to the first element of A. We claim that **at the beginning of the i-th iteration**, `currentMax` equals the maximum of the first i elements.

Since we compare `currentMax` to `A[i]` in iteration i — if this claim is true before the iteration, it will be true after it for i+1. Thus, after n−1 iterations, `currentMax` equals the maximum element in A.

```
Algorithm arrayMax(A, n):
  Input:  array A, n ≥ 1
  Output: maximum element

  currentMax ← A[0]
  for i ← 1 to n - 1 do
     if currentMax < A[i] then
        currentMax ← A[i]
  return currentMax
```

Pseudocode Constructs

- ***Expressions:*** Standard mathematical symbols.
  Assignment uses `←` (equivalent to `=` in C++). Equality uses `=` (equivalent to `==` in C++).
- ***Method declarations:*** `Algorithm name(param1, param2, ...)`
  declares a new method and its parameters.
- ***Decision structures:*** `if condition then true-actions [else false-actions]`.
  Indentation indicates scope.
- ***While-loops:*** `while condition do actions`.
- ***Repeat-loops:*** `repeat actions until condition`.
- ***For-loops:*** `for variable-increment-definition do actions`.
- ***Array indexing:*** `A[i]`
  usually indexed from 1 to n (math convention) or 0 to n−1 (C/C++/Java).
- ***Method calls:*** `object.method(args)`. AND ***Method returns:*** `return value`.

THE RANDOM ACCESS

Machine (RAM) Model

THE RAM MODEL — MOTIVATION

If we wish to analyze a particular algorithm without performing experiments on its running time, we can take an **analytic approach** directly on the high-level code or pseudocode.

We define a set of high-level **primitive operations** that are largely independent from the programming language used and can be identified also in the pseudocode.

THE SEVEN PRIMITIVE OPERATIONS

Symbol	Operation	Example
c_1	Assigning a value to a variable	`x = 5`
c_2	Calling a method	`arrayMax(A, n)`
c_3	Performing an arithmetic operation	`a + b`, `i++`
c_4	Comparing two numbers	`currentMax < A[i]`
c_5	Indexing into an array	`A[i]`
c_6	Following an object reference	`node->next`
c_7	Returning from a method	`return currentMax`

THE RAM MODEL — DEFINITION

This approach of counting primitive operations gives rise to a computational model called the **Random Access Machine (RAM)**. This model — which should **not** be confused with "random access memory" — views a computer simply as a CPU connected to a bank of memory cells.

Each memory cell stores a word: a number, a character string, or an address. The term ***random access*** refers to the ability of the CPU to access an arbitrary memory cell with **one primitive operation**.

We assume the CPU in the RAM model can perform any primitive operation in a **constant number of steps**, which do not depend on the size of the input. Thus, an accurate bound on the number of primitive operations an algorithm performs corresponds directly to the running time of that algorithm in the RAM model.

THE RAM MODEL — WHY CONSTANT TIME?

Specifically, a primitive operation corresponds to a low-level instruction with an execution time that depends on the hardware and software environment but is **nevertheless constant**.

Instead of trying to determine the specific execution time of each primitive operation, we will simply **count** how many primitive operations are executed, and use this number **t** as a high-level estimate of the running time of the algorithm.

The implicit assumption in this approach is that the running times of different primitive operations will be **fairly similar**. Thus, the number **t** of primitive operations an algorithm performs will be **proportional** to the actual running time of that algorithm.

COUNTING

Primitive Operations

COUNTING PRIMITIVE OPERATIONS — SETUP

```
Algorithm arrayMax(A, n):
  Input:  array A, n ≥ 1
  Output: maximum element

  currentMax ← A[0]
  for i ← 1 to n - 1 do
      if currentMax < A[i] then
          currentMax ← A[i]
  return currentMax
```

- c_1 Assigning a value to a variable
- c_2 Calling a method
- c_3 Performing an arithmetic operation
- c_4 Comparing two numbers
- c_5 Indexing into an array
- c_6 Following an object reference
- c_7 Returning from a method

STEP 1 — INITIALIZING CURRENTMAX

```
currentMax ← A[0]
```

Initializing `currentMax` to `A[0]` corresponds to **two** primitive operations:

- c_5 — indexing into array A at position 0
- c_1 — assigning the value to `currentMax`

Executed only **once** at the beginning.

$$2+$$

- c_1 Assigning a value to a variable
- c_2 Calling a method
- c_3 Performing an arithmetic operation
- c_4 Comparing two numbers
- c_5 Indexing into an array
- c_6 Following an object reference
- c_7 Returning from a method

STEP 2 — INITIALIZING LOOP COUNTER

```
for i ← 1 to n - 1 do
```

```
for (int i = 1; ...
```



At the beginning of the `for` loop, counter `i` is initialized to 1. This corresponds to **one** primitive operation:

- c_1 — assigning value 1 to variable i

Executed only **once**.

$$2 + 1 +$$

- c_1 Assigning a value to a variable
- c_2 Calling a method
- c_3 Performing an arithmetic operation
- c_4 Comparing two numbers
- c_5 Indexing into an array
- c_6 Following an object reference
- c_7 Returning from a method

STEP 3 — LOOP CONDITION CHECK

```
for i ← 1 to n - 1 do
```

```
for (int i = 1; i < n; i++)
```



Before entering the body of the loop, condition `i < n` is verified. This corresponds to **one** primitive operation:

- c_4 — comparing two numbers

Since `i` starts at 1 and is incremented by 1, the comparison `i < n` is performed **n**

- c_1 Assigning a value to a variable
- c_2 Calling a method
- c_3 Performing an arithmetic operation
- c_4 Comparing two numbers
- c_5 Indexing into an array
- c_6 Following an object reference
- c_7 Returning from a method

STEP 4 — LOOP BODY

The loop body executes **n-1 times**.

At each iteration:

- `A[i]` compared with `currentMax` → **c_5 + c_4** = 2 ops
- `i++` — increment counter → **c_3 + c_1** = 2 ops
- `currentMax ← A[i]` if condition true → **c_5 + c_1** = 2 ops *(conditional)*

4 ops if `A[i] ≤ currentMax` (no reassignment)

6 ops if `A[i] > currentMax` (reassignment

- **c_1** Assigning a value to a variable
- **c_2** Calling a method
- **c_3** Performing an arithmetic operation
- **c_4** Comparing two numbers
- **c_5** Indexing into an array
- **c_6** Following an object reference
- **c_7** Returning from a method

STEP 4 — TWO CASES

Worst case — every element larger than previous (sorted ascending):

`currentMax` reassigned at every iteration → **6 ops** per iteration

$$2 + 1 + n + (n-1)(2+2+2) = 2 + 1 + n + (n-1)(6)$$

Best case — first element is the maximum:

`currentMax` never reassigned → **4 ops** per iteration

$$2 + 1 + n + (n-1)(2+2) = 2 + 1 + n + (n-1)(4)$$

STEP 5 — RETURN STATEMENT

```
return currentMax
```

Returning the value of `currentMax` corresponds to **one** primitive operation:

- ⚒ c_7 — returning from a method

Executed only **once**.

Worst case:

$$2 + 1 + n + (n-1)(6) + \mathbf{1}$$

Best case:

$$2 + 1 + n + (n-1)(4) + \mathbf{1}$$

- ⚒ c_1 Assigning a value to a variable
- ⚒ c_2 Calling a method
- ⚒ c_3 Performing an arithmetic operation
- ⚒ c_4 Comparing two numbers
- ⚒ c_5 Indexing into an array
- ⚒ c_6 Following an object reference
- ⚒ c_7 Returning from a method

FINAL COUNT — T(N)

```
Algorithm arrayMax(A, n):
  currentMax ← A[0]
  for i ← 1 to n - 1 do
     if currentMax < A[i] then
        currentMax ← A[i]
  return currentMax
```

Best case — A[0] is the maximum:

$$2 + 1 + n + (n-1)(4) + 1$$
$$= 3 + n + 4n - 4 + 1 = \mathbf{5n}$$

Worst case — array sorted ascending:

$$2 + 1 + n + (n-1)(6) + 1$$
$$= 3 + n + 6n - 6 + 1 = \mathbf{7n - 2}$$

> Best case: T(n) = 5n — A[0] is maximum, `currentMax` never reassigned.
> Worst case: T(n) = 7n − 2 — sorted ascending, reassigned at every step.

FROM T(N) TO ASYMPTOTIC NOTATION

Both the best case and the worst case give a T(n) that is **linear in n**:

$$5n \leq T(n) \leq 7n - 2 \quad \text{for all } n \geq 1$$

This means we can find constants c_1 and c_2 such that:

$$c_1 \cdot n \leq T(n) \leq c_2 \cdot n \quad \text{for all } n \geq n_0$$

Therefore:

$$T(n) = \Theta(n)$$

> The exact constants (5 and 7) are irrelevant for the asymptotic class.
> Both best and worst case are **linear** — T(n) = Θ(n).

AVERAGE-CASE AND

Worst-case Analysis

AVERAGE-CASE AND WORST-CASE ANALYSIS

Like the `arrayMax` method, an algorithm may run faster on some inputs than others. We may wish to express running time as an **average** taken over all possible inputs.

Although such an **average-case analysis** would often be valuable, it is typically quite challenging. It requires us to define a **probability distribution** on the set of inputs — which is typically a difficult task.

AVERAGE-CASE AND WORST-CASE ANALYSIS



Running time varies between best-case and worst-case depending on the input.

What if in practice inputs are only of type "A" or "D"?

WHY WE USE WORST-CASE ANALYSIS

- Average-case analysis typically requires calculating **expected running times** based on a given input distribution. Such an analysis often requires heavy **mathematics and probability theory**.
- Therefore, we will typically characterize running times in terms of the **worst case**. We say that `arrayMax` executes **t(n) = 7n − 2** primitive operations **in the worst case** — meaning the maximum number of primitive operations over all inputs of size n.
- This type of analysis is much **easier** than average-case. It does not require probability theory — just the ability to identify the worst-case input, which is often straightforward.
- Designing for the worst case leads to stronger algorithmic "muscles" — **much like a track star who always practices by running uphill**.

Which Is Faster? — Back to the Curio

```c
// Version 1 — if statement
if (currentMax < A[i])
    currentMax = A[i];
// Version 2 — ternary operator
currentMax = currentMax < A[i] ? A[i] : currentMax;
```



In the **worst case** both versions execute the same primitive operations — the ternary operator always evaluates both branches conceptually.

In the **best case** Version 1 skips the assignment when $A[i] \leq currentMax$.
Version 2 **always** performs the assignment — even when value does not change.

In the RAM model both are **Θ(n)**. The difference is only in the constant — which disappears asymptotically.

> The asymptotic class is the same. The constant factor differs — and only matters in practice at very large scale.

THANK

You